Stanley Nelavala

Software Engineer: Backend, Platform, Full Stack and AI

218-213-8220 | consultantstanleymn@gmail.com | LinkedIn | GitHub | stanley-n.com
Huntington, IN | Open to relocate anywhere

Summary

Software engineer who owns systems end to end. At a 10-person hospital operations startup I designed the service-to-service API that every platform module uses, built the hospital design service and the iOS and Android asset app, cut a deep catalog query from 285 ms to 0.20 ms, and cut dev and test server runtime from 168 to about 53 hours a week. Public projects in Python, Java, C#, Go and Rust with tests, SLOs and failure labs. M.S. in Computer Science (4.0 GPA). AWS Certified Solutions Architect, Associate.

Experience

Software Development Engineer Nov 2025 – Present
Mass Technologies, team of 10 (RxApps360 hospital platform, pre-launch; numbers from dev and test) *Huntington, IN*

- Designed, built and integrated into every platform module the signed service-to-service API (M2M): each call proves which service and tenant sent it. After a security audit, hardened it with a Redis rate limiter and replay protection.
- Built RxDesign, the hospital design service, backend and web (FastAPI, React, TypeScript), with a deterministic, rule-based layout engine that places rooms on a 0.5 ft grid with doors, exits and flow lines; 220+ offline tests on the API.
- Built the RxAsset iOS and Android app (React Native, Expo) with scan-to-search and an on-device voice assistant (speech-to-text and text-to-speech run on the phone, no cloud call); fixed a native crash in the speech library bridge.
- Set up separate iOS and Android release pipelines in GitHub Actions with automatic TestFlight submission.
- Replaced offset paging with keyset paging on the catalog device list: on a 328,000-device dev database, the query for row 150,000 fell from 285 ms to 0.20 ms. Search pages never skip or repeat rows, tested with rows inserted, deleted and tied between pages.
- Cut the planning Items page from 3 API calls and 162 KB to 2 calls and 37 KB with typeahead search and cursor paging.
- Fixed a regular expression denial-of-service bug in a shared validation package: an 80-character hostile input took about 138 s; the new linear pattern matched the old results on 60,000 random inputs.
- Automated weekday start and stop of dev and acceptance servers and databases (AWS ECS, RDS): they run about 53 hours a week instead of 168.
- Standardized malware scanning of file uploads (AWS GuardDuty) across 9 app modules.
- Resolved 27 defects across the asset transfer and parts pages.

Adjunct Faculty (part-time) Jan 2026 – Aug 2026
The College of St. Scholastica *Duluth, MN*

- Taught Programming II (Java) and Machine Learning for Data Science.

Graduate Research Assistant and Teaching Assistant Aug 2023 – Aug 2025
University of Minnesota Duluth *Duluth, MN*

- Built a pipeline (Python, Node.js, MySQL, FFmpeg, PaddleOCR) that collected and analyzed 1M+ TikTok videos from 300k+ authors for thesis research on misinformation, then grouped themes with BERTopic; presented at the UMD CS graduate poster session. Led labs and office hours for Python, NLP and Parallel Programming.

Projects (public code, tests)

Agent Integration Platform | *Python, FastAPI, LangGraph, MCP, PostgreSQL (pgvector), Redis, Terraform* GitHub

- LangGraph case workflow with typed state, a Postgres checkpointer, human approval that resumes after restart, and step and token budgets; one constrained MCP write tool.
- 62 deterministic replay evals, prompt-injection tests, output guardrails, PII redaction and an append-only audit log.
- Hybrid retrieval (pgvector plus full-text search fused by RRF) with access labels; failover between two LLM adapters with timeouts, a retry budget, a circuit breaker and token cost records.

Payments Ledger Service | *Java 25, Spring Boot, PostgreSQL, Flyway, Kafka API, Prometheus, Grafana, k6* GitHub

- Double-entry ledger (authorize, capture, void, refund, transfer) with idempotency keys and a transactional outbox to a Kafka API (Redpanda); property tests and a k6 check that no money is created or lost.
- Three user-facing SLIs with 28-day error budgets and six multi-window burn-rate alerts tested with promtool; a scripted failure lab (Kafka down, Postgres restart, relay crash, poison event, latency injection) with balance checks after each.
- Rolled forward and back by image digest across an expand-and-contract Flyway migration; a k6 SLO gate fails a forced slow run; dead-letter topic with once-only redrive.

Ledger Ops Console and QA Suite | *React, TypeScript, Playwright, axe, Cucumber, Selenium, JMeter* GitHub

- React 19 and TypeScript ops console over read-only endpoints that share one keyset paging contract: 32 Vitest and 13 Playwright tests, 6 axe tests with zero WCAG 2.1 A and AA violations.
- Separate Java QA suite: 41 black-box API tests (16 Cucumber scenarios, REST Assured and JSON-schema contract tests), 6 Selenium UI tests, 0 flaky; JMeter gate passed with about 32,000 samples, 0% errors, p95 81 to 148 ms against 300 ms (local laptop numbers).

Card Dispute Service | *C#, .NET 10, ASP.NET Core, EF Core, PostgreSQL, xUnit, CloudFormation* GitHub

- Dispute state machine where concurrent decisions on one case produce exactly one winner (PostgreSQL xmin optimistic concurrency); idempotent create and keyset paging tested with ties, inserts and deletes; tests written first (30 unit, 32 integration).

- Non-root container image with a clean Trivy scan, Prometheus metrics and readiness checks; CloudFormation for ECS Fargate linted and policy-checked only, with no deploy run.

Inference Gateway | *Go, gRPC, ONNX Runtime, PyTorch, OpenTelemetry, Prometheus, Helm* GitHub

- Go service for an int8-quantized banking-intent classifier over HTTP and gRPC with micro-batching, deadlines and load shedding; macro-F1 0.8636 for both fp32 and int8 on the BANKING77 test split.
- Canary controller with a weighted proxy and an error and p95 gate that rolled a faulty version back on its own; latency SLO alerts tested with promtool; Helm chart tested on kind.
- One-factor experiments and pprof ruled out GC and tokenizer cost; a Little's law model sizes a 10x launch peak at 2 to 3 replicas (local benchmark).

Webhook and Connector Gateway | *Rust, axum, tokio, PostgreSQL, SQLite, sqlx, OpenTelemetry, k6* GitHub

- Rust gateway: HMAC-SHA256 webhooks verified in constant time, per-tenant rate limits and idempotency, durable store before delivery, retries with backoff, dead letters with replay and circuit breakers.
- Postgres store (FOR UPDATE SKIP LOCKED claim) beside SQLite behind one store port; the same 88 tests pass on both backends.
- Global backpressure (503 with Retry-After) and a kill -9 failure lab where 500 of 500 events reached the downstream once after redrive; delivery SLO alerts tested with promtool.

Customer Integration Console | *TypeScript, Next.js, Fastify, PostgreSQL, OIDC, Trivy* GitHub

- OIDC discovery with PKCE against a mock IdP and client-credentials scopes for machine clients; an authorization matrix test over every route and role and 10 negative token tests.
- Envelope encryption (AES-256-GCM) of source credentials with key rotation, a hash-chained audit log with tamper detection, CSP and security headers; 256 tests plus 7 browser tests.

FHIR Analytics Pipeline | *Python, Airflow, dbt, DuckDB, Azure Blob SDK (Azurite), Bicep, HL7 FHIR R4* GitHub

- dbt model contracts, source freshness, a quarantine-rate gate and a provenance manifest per run; k-anonymity check (k=5) and versioned HMAC pseudonymization keys with a rotation test; 108 tests.

Aurelia, native iPhone voice tutor | *SwiftUI, Speech, SQLite, Swift Package Manager* GitHub

- Built and signed on Fedora with no Mac; 152 tests pass with swift test on Linux; spaced repetition ladder and a token stream with interleaved whiteboard tags.

Other projects

Mina (private repo): invoicing and purchases web app (FastAPI, PostgreSQL, React, TypeScript) with exact integer money math, PDF export, forced row-level security, an audit chain and about 7,700 backend tests; no users.
Live Defined (local): redesign and extension of a supplied Next.js and Payload CMS codebase with a host portal and light and dark themes; about 3,700 unit tests; no users.
openGym for iPhone (public): native iPhone build from a Fedora laptop with no Mac, wrapping the AGPL openGym app. **Study Mentor** (public): self-hosted voice tutor (FastAPI, faster-whisper, Kokoro).
Local projects: Trade Settlement and Reconciliation Platform (Java, Kotlin, Spring Boot, Oracle, JMS, Kafka Streams, Spring Batch, Angular; 27 Cucumber scenarios), Platform Golden Path (Helm, Kustomize, Jenkins shared library with 19 Groovy tests), KYC Onboarding (Python, AWS CDK synth, Step Functions Local, 78 unit tests).

Skills

Languages: Python, TypeScript, JavaScript, SQL, Java, C#, Go, Rust, Kotlin, Swift, Bash
Backend and data: FastAPI, Spring Boot, ASP.NET Core, EF Core, Fastify, axum, REST, gRPC, PostgreSQL, Redis, Kafka API (Redpanda), SQLite, migrations (Alembic, Flyway, sqlx), Airflow, dbt, DuckDB
Frontend and mobile: React, Next.js, React Native, Expo, Sass, Vite, SwiftUI, accessibility (axe, WCAG 2.1 A and AA)
AI and ML: LangGraph, MCP, hybrid retrieval and RAG, deterministic LLM evals, guardrails, PyTorch, ONNX Runtime, BERTopic, on-device speech; AI-assisted development (Claude Code, Codex, opencode, DeepSeek)
Cloud and delivery: AWS (ECS, RDS, S3, SQS, Secrets Manager, GuardDuty), Terraform, CloudFormation (lint), Docker, Kubernetes (kind), Helm, GitHub Actions
Reliability and security: OpenTelemetry, Prometheus, Grafana, SLOs and burn-rate alerts, failure labs, canary rollback, STRIDE, OIDC, envelope encryption, Trivy, gitleaks
Testing: pytest, Jest, Vitest, Playwright, JUnit 5, Testcontainers, xUnit, Cucumber, REST Assured, Selenium, JMeter, k6, Locust

Education and Certification

M.S. Computer Science, University of Minnesota Duluth, 2023 – 2025, GPA 4.0, Graduate Summer Research Fellowship 2024 and 2025
B.Tech Computer Science and Engineering, IIIT D&M Kancheepuram, 2019 – 2023
AWS Certified Solutions Architect, Associate, Feb 2025 – Feb 2028 (verifiable on Credly)